

01 / THE PROJECT

Understand Vault.

A practical guide for teammates: what we built, how requests move through the system, and what its guarantees actually mean.

DISTRIBUTED OBJECT STORAGE

Store. Replicate. Verify. Recover.

One logical object, multiple persistent copies, and a consistent record of which version is current.

An object is a file plus metadata. Its bucket groups related objects; its key names it. Vault stores the bytes on storage nodes and keeps the current version and replica locations in a metadata service.

Console

The interface people use to upload files and inspect the cluster.

Gateway

The coordinator that authenticates, places, publishes and repairs data.

Storage + metadata

Disks hold bytes. The metadata service identifies the committed version.

Current status

A working distributed MVP with a live dashboard, persistent replicas and failure tests. This guide distinguishes implemented behavior from unverified production guarantees.

READING ROUTE	PAGES
Stack and component architecture	2-3
Uploads, reads, consistency and recovery	4-6
Security, operating limits and evidence	7-8
Run locally, find code, learn the vocabulary	9-10

SOURCE README.md; ARCHITECTURE.md; project version in package.json

02 / TECHNOLOGY

The stack, layer by layer

The application services are written in JavaScript and TypeScript. Node.js runs the backend; etcd supplies the consensus metadata service.

LAYER	TECHNOLOGY	WHAT IT DOES
Languages	TypeScript 5.7.3 JavaScript ES modules	Typed dashboard and simulator; .mjs gateway and storage services.
Web application	Next.js 16.3.6 React 19 + React DOM	Pages, interactive console, session endpoint and streaming API proxy.
Design + UI	Tailwind CSS 4, PostCSS Custom CSS, shadcn/ui, Base UI	Responsive styling and shared UI primitives. The durable console uses custom components.
Client utilities	SWR, Fetch API Lucide React, Sonner	Polling and cache refresh, HTTP requests, icons and notifications.
Backend runtime	Node.js 22 Native HTTP / HTTPS	Independent gateway and storage processes, with REST-style APIs.
Data movement	Node streams + pipelines Web Streams	Buffered streaming, temporary-file staging and replica transfers.
Object bytes	Filesystem replica files JSON receipts	Persistent immutable payloads, checksums and durable write receipts.
Metadata	etcd 3.6.5 / Raft Local transaction journal	Consensus transactions in etcd mode; single-process journal for local development.
Integrity	SHA-256, Node crypto Browser Web Crypto	Object and chunk checksums, random credentials and upload-part hashing.
Build + quality	npm 10.9.3, Turbopack ESLint, Prettier, TypeScript	Locked dependencies, bundling, lint, formatting and type checks.
Testing + delivery	node:test, node:assert Docker / Compose, GitHub Actions	Process-failure tests; container and CI configurations are supplied.

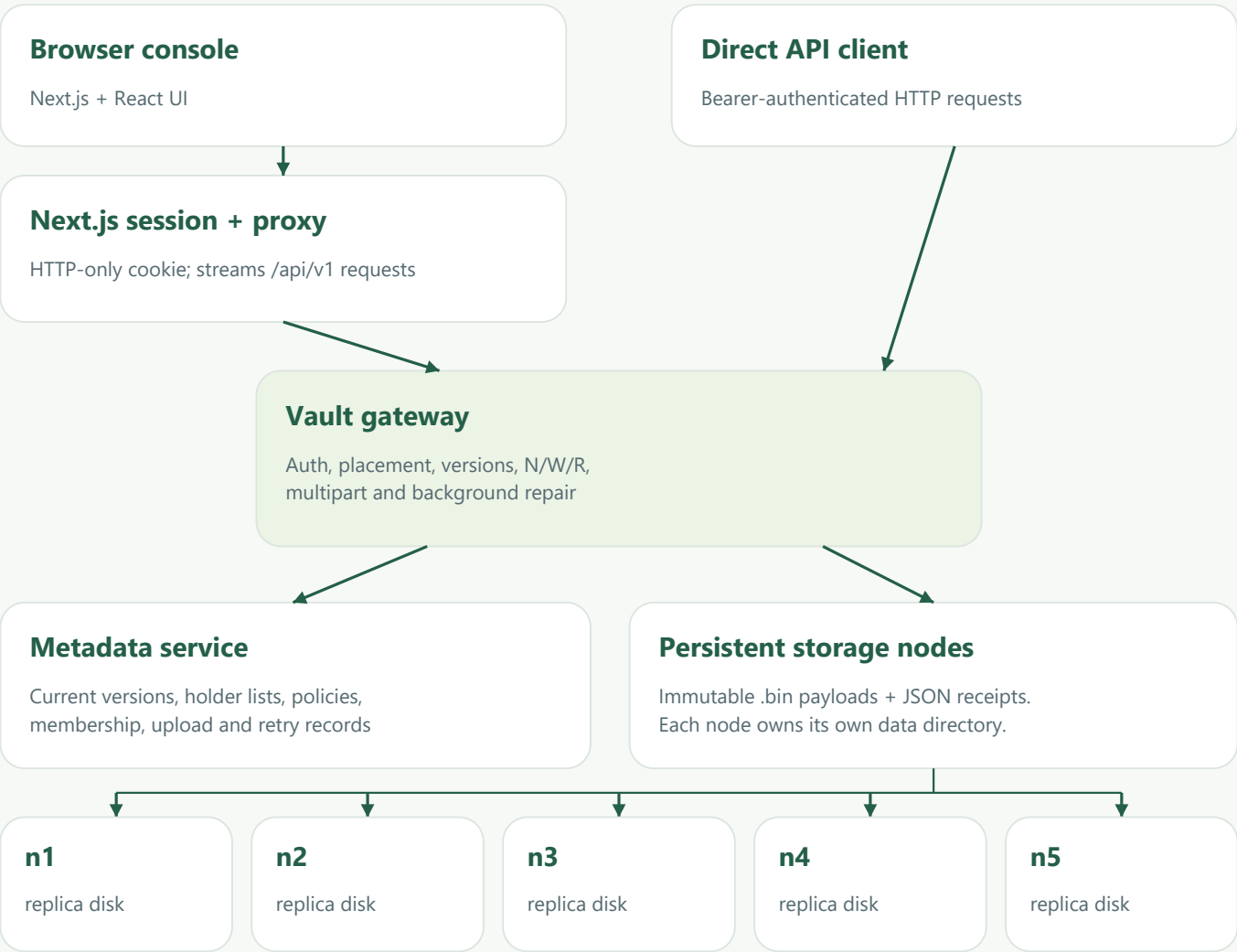
Supporting packages: class-variance-authority, clsx, tailwind-merge, cn and tw-animate-css. next-themes is used by a retained UI wrapper. @vercel/analytics remains installed but is not enabled in the current dashboard.

Why this stack? It builds on the existing Next.js project, supports streamed network and disk I/O, and keeps hackathon integration fast. This is a practical choice, not benchmark evidence that Node.js outperforms Go or Java.

03 / ARCHITECTURE

Where each responsibility lives

The console is the user interface. The gateway coordinates requests. Storage nodes own payload files; metadata owns the current object pointer.



MODE	METADATA + DEPLOYMENT
Local development	Five storage processes + one gateway; a disk-backed local journal. Persistent, but all services share one machine.
etcd topology	Three etcd members + five storage services in Compose. The implementation supports multiple gateways; the supplied Compose file starts one.

Separate bytes from decisions. etcd stores metadata only, not uploaded file contents. Losing the metadata majority blocks authoritative reads and writes. Five containers on one laptop do not provide five independent hardware failure domains.

SOURCE services/gateway.mjs; services/storage.mjs; services/metadata.mjs; compose.yaml

04 / UPLOAD PATH

When is a write really saved?

An upload becomes visible only after durable replica acknowledgements and a successful metadata transaction.

1

Authorize and read current state

Check the role and bucket; read policy, membership and the current object revision.

2

Stage and hash the incoming bytes

Stream to a gateway temporary file; compute size, full SHA-256 and 1 MiB chunk hashes.

3

Choose replica destinations

Rendezvous hashing ranks nodes; placement selects distinct configured failure domains.

4

Persist replicas and collect acknowledgements

Each node verifies bytes, syncs its file and publishes a durable receipt. Require at least W acknowledgements.

5

Publish the committed version atomically

Compare object and configuration revisions, then record the object pointer and idempotency result together.

6

Return the result to the client

A failed quorum or competing update does not publish a partially stored version. A timeout can still have an unknown outcome.

Large-file uploads can resume

The browser uses multipart above 8 MiB, normally in 8 MiB parts. Parts are replicated and their state persists. Reselecting the same file resumes missing parts within a 24-hour session. Completion assembles the ordered parts and publishes the final object atomically.

Retry rule: keep the same Idempotency-Key for the same operation. If the server committed but the response was lost, the recorded transaction result resolves the retry. Failed uploads may leave unreferenced files until safe physical cleanup is implemented.

SOURCE `services/gateway.mjs: writeObject` and `multipart`; `services/storage.mjs: PUT replica`

05 / CONSISTENCY

Reads, versions and N/W/R

Metadata selects the current version. Replica policy decides whether enough matching copies are reachable; a checksum decides whether bytes are intact.

1

Look up the committed object

Read the authoritative metadata pointer; a deletion marker means the object is absent.

2

Check replicas, then verify a download

Require the configured availability replies. Try eligible copies and stage a full SHA-256-verified payload.

3

Serve verified bytes

Only then return the full file or requested byte range. Full-file staging adds latency and temporary disk usage.

POLICY	PLAIN-LANGUAGE MEANING	DEFAULT
N	Desired number of replica copies in distinct failure domains.	3
W	Durable replica acknowledgements needed to publish a PUT.	2
R	Matching availability replies needed for a normal GET. These replies alone do not prove integrity.	2

Example: N=3, W=2, R=2. If two intact copies remain reachable, a normal read can succeed. If only one remains, the normal read fails, but an explicit consistency=one request may recover that verified copy. Metadata quorum must still be available in both cases.

SITUATION	IMPLEMENTED BEHAVIOR
Two writers race	Compare-and-swap publication prevents both from committing against the same prior revision. The loser can receive 409.
Client updates a known version	If-Match protects that version; If-None-Match: * requires absence. Failed preconditions return 412.
Deletion or an old replica reappears	A persistent metadata tombstone remains authoritative. Repair does not promote stale bytes into a current object.

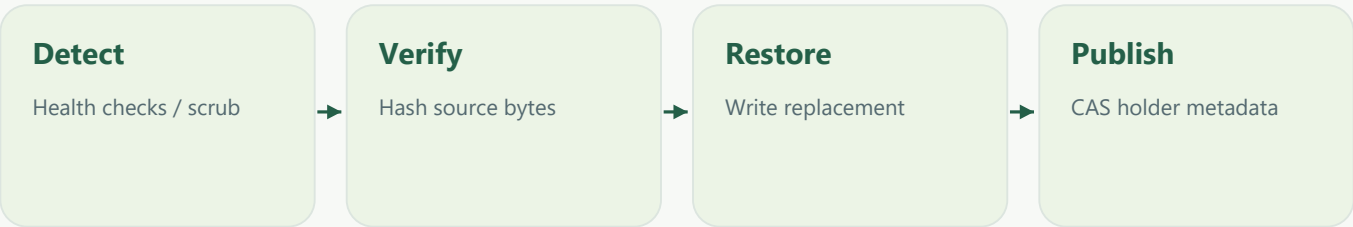
The ETag identifies the committed version, not its content hash. Concurrent reads may observe the version current when their metadata lookup occurred. $R + W$ arithmetic is not the source of metadata consistency here.

SOURCE ARCHITECTURE.md; API.md; services/gateway.mjs; services/metadata.mjs

06 / FAILURE HANDLING

Detect, verify, restore

Background work compares the current metadata with actual node health and replica integrity. It uses intact sources to rebuild missing or corrupt copies.



FAILURE / CHANGE	RESPONSE	BOUNDARY
Storage process stops	Healthy replicas may serve reads; repair can rebuild copies on spare domains.	Read availability still depends on R and surviving data.
Payload bytes are corrupted	Read verification rejects corrupt bytes; scrub and repair locate an intact source.	Repair currently copies the whole object.
A node is partitioned or slow	RPC deadlines mark failed operations; fallback placement is policy-controlled.	Real cross-host partition and slow-network testing remain required.
etcd leader fails	A surviving metadata majority can elect a new leader and continue.	Losing the majority blocks metadata-authorized reads and writes.
Node joins or placement changes	Reconciliation copies current objects toward desired placement.	Old files remain; physical space is not automatically reclaimed.
Operator drains a node	Exclude new placement; verify N replacement domains before removing holder references.	Removal also waits for current references and active multipart sessions.

Replica repair is different from backup

Replication protects against some node failures; it does not provide a separate historical recovery system. Backups must preserve metadata, storage volumes and the mapping between node IDs and disks.

Storage tradeoff: with N=3, current object replicas alone use roughly three times the logical data size. Multipart parts, old versions, failed writes and temporary staging add overhead. Safe garbage collection is still a production gap.

SOURCE services/gateway.mjs: repairTick; services/storage.mjs: verify / scrub; OPERATIONS.md

07 / OPERATIONS

Access, protection and limits

These controls exist in the implementation. Public deployment still needs trusted networking, certificate management, operational monitoring and capacity planning.

CONTROL	WHAT THE CURRENT SYSTEM DOES
Client access	Bearer tokens with admin, write and read roles; authorization can be restricted to selected buckets.
Browser session	HTTP-only, SameSite=Strict cookie; origin checks on mutations. Secure cookies are configurable for HTTPS.
Service access	A separate internal node token authenticates gateway-to-storage requests. Credentials are compared in constant time.
Transport	Native HTTP or HTTPS. TLS requires certificate/key configuration; trusted internal CAs can be supplied to Node clients.
Visibility	Health endpoints, activity events, JSON audit logs and Prometheus-compatible counters. External collection/retention must be configured.

Key defaults to understand

SETTING	DEFAULT	WHY IT MATTERS
Object size limit	1 GiB	Configurable; this limit is not a demonstrated throughput guarantee.
Checksum chunk size	1 MiB	Chunk digests are recorded; download validation checks the whole object.
Concurrent requests	32 per service	Bounds in-flight work; excess load can return 503.
Replica RPC deadline	30 seconds	Includes streamed bodies; large objects and slow links require tuning.
Repair cycle / budget	3 seconds / 64 MiB	Up to eight object records per pass; one object can exceed the byte budget.
Node integrity scrub	One replica / 2 seconds	Works incrementally; a complete scan takes longer as inventory grows.
Multipart API part limit	16 MiB	The browser normally sends 8 MiB parts; sessions last 24 hours.

Durability detail: files are synced before acknowledgement. Linux can also sync directory changes; Windows local tests do not establish that same power-loss guarantee. Credentials are supplied at startup and require coordinated rotation.

SOURCE `services/common.mjs`; `services/storage.mjs`; `services/gateway.mjs`; `app/api/session`; `OPERATIONS.md`

08 / EVIDENCE

What we can demonstrate

Verification was recorded on 25 September 2026 using Windows, Node.js 22.20.0 and real etcd 3.6.5 binaries. The guide was prepared on 26 September; tests were not rerun for this document.

12

tests passed; no skips

9 MiB+

browser upload verified

2

gateways tested with etcd

VERIFIED AREA	EVIDENCE IN THE REPOSITORY
Data path	File larger than the old 8 MiB cap; full/range reads; browser upload and byte-identical download of 9,437,201 bytes.
Correctness under changes	Conditional-write races, idempotency, permissions, bucket scopes, policy changes, node registration and draining.
Failure recovery	Actual on-disk byte corruption, killed processes, failed write quorum, spare-node repair, multipart restart and complete-cluster restart.
Consensus metadata	Three real etcd processes; two gateways, conflicting writes, retry replay, leader loss, majority loss and durable restart.
Build and delivery checks	Production build, TypeScript, lint and formatting passed. Compose configuration was validated statically.

Remaining production acceptance gates

- 1. **Bound storage growth:** safe physical garbage collection, retention rules and metadata compaction/indexing.
- 2. **Measure scale:** sustained load, soak tests, resource limits and recovery objectives.
- 3. **Validate real failure domains:** multi-host partitions, disk exhaustion and Linux power-loss behavior.
- 4. **Exercise operations:** backup/restore, rolling upgrades, gateway failover, certificate and secret rotation.
- 5. **Run the containers:** Docker runtime testing was unavailable because its daemon was stopped.

What the results do not establish

Passing correctness tests is not proof of unlimited scale, bounded recovery time or production readiness. Erasure coding, S3 compatibility, encryption at rest/key management and multi-region operation are outside this version.

SOURCE VERIFICATION.md; test/integration.test.mjs; test/metadata.test.mjs; test/etcd.test.mjs

09 / TEAM ONBOARDING

Run it and find the code

Start with local mode for development. Use etcd mode when testing consensus behavior. The README contains the full setup steps.

TERMINAL	COMMANDS / RESULT
1 - storage cluster	<code>npm ci</code> <code>npm run vault:dev</code> Starts five storage processes on 7401-7405 and the gateway on 7400.
2 - dashboard	<code>npm run dev</code> Open <code>http://127.0.0.1:3000</code> in the browser.
3 - sign-in credential	<code>npm run vault:token</code> Use the generated token locally. Never share it in screenshots or commits.

Other workflows: `npm run build` then `npm start` runs the optimized console. `npm test` runs the core suite; set `ETCD_BINARY` to a real etcd executable to include consensus tests. Docker setup: `node scripts/configure-compose.mjs`, then `docker compose up --build -d`.

Code ownership map

FILE / DIRECTORY	RESPONSIBILITY
<code>services/gateway.mjs</code>	Object APIs, placement, quorum policy, multipart, publication and repair.
<code>services/storage.mjs</code>	Durable replica writes, reads, receipts, node health and integrity scrubbing.
<code>services/metadata.mjs</code>	etcd API transactions and the local metadata journal.
<code>services/common.mjs</code>	Streaming, checksums, filesystem durability, locks, deadlines and TLS.
<code>components/vault/ durable-dashboard.tsx</code>	Live operator console: objects, cluster, policies, activity and uploads.
<code>app/api/session</code> + <code>app/api/v1</code>	Browser authentication and the streaming gateway proxy.
<code>scripts/</code> + <code>test/</code>	Local launchers, deployment helpers and automated verification.
<code>lib/vault/</code>	Retained simulator; active only with <code>VAULT_MODE=simulator</code> .

Local development has a deliberate boundary

The launcher stops the other local services if one child exits. For independent crash demonstrations, use the automated suite or a validated Compose deployment. Local data lives in `.vault/`; the lightweight source ZIP excludes runtime data and credentials.

SOURCE README.md; package.json; scripts/dev-cluster.mjs; project source tree

10 / SHARED VOCABULARY

Explain the design consistently

Use these meanings during implementation discussions and the presentation. Keep the original simulator and the persistent backend distinct.

TERM	WHAT IT MEANS IN VAULT
Object / bucket / key	A file with metadata / a namespace grouping / the name identifying an object within that namespace.
Replica / failure domain	A stored copy / a boundary expected to fail independently, such as a separate host or disk.
Manifest / receipt	Committed object metadata / a storage node record describing a durably stored replica.
Quorum / consensus	An acknowledgement threshold / agreement among metadata members about committed state.
CAS / idempotency	Publish only if the previous revision still matches / safely resolve retries of the same operation.
Checksum / scrub	A digest used to detect altered bytes / a background integrity scan of stored data.
Tombstone / repair	A durable deletion marker / reconstruction of missing or corrupt replica copies.
Drain / rebalance	Safely retire a node / reconcile replica placement after membership or policy changes.

Two implementations remain in the repository

PERSISTENT BACKEND	OPTIONAL ORIGINAL SIMULATOR
Independent Node.js processes, disk files, rendezvous placement, etcd or local journal, CAS versions, verified whole-object repair.	In-memory TypeScript model with a consistent-hash ring, simulated Raft, hybrid clocks, last-write-wins, hinted handoff and Merkle comparisons.

Language choice: the application backend uses Node.js/JavaScript, not Spring Boot, Java, C++ or Maven. etcd is an external Go-based service. CPU-heavy work can contend with the Node event loop; a language change would require workload evidence, not just preference.

Project references

README.md - setup and scope. **ARCHITECTURE.md** - publication and consistency contract. **API.md** - routes, headers and errors. **OPERATIONS.md** - deployment, capacity and recovery. **VERIFICATION.md** - recorded test evidence. **DEMO.md** - presentation walkthrough. **package.json** / **package-lock.json** - dependencies and scripts.

This guide is grounded in the Vault v0.2.0 source at vault-distributed-storage-design. Implementation and test evidence take precedence if this document and later code changes diverge.

SOURCE Repository files listed above; lib/vault/ versus services/